

Daily Progress Report

Week 4_Day3: Async / Await & Error Handling

Program	3-Month MERN Stack Internship Program
Intern Name	Mohsin Ali
Week	Week_4 (Day 3)
Date	15 July, 2026

Day 3: Async / Await & Error Handling

1.1 Topics Studied

- Refactoring Promise-based code into modern async/await syntax
- Understanding the execution flow of asynchronous functions
- Implementing robust error handling with try, catch, and finally blocks
- Managing operational failures without disrupting the main execution thread
- Designing user-centric fallback UI for error states
- Advanced DOM manipulation for dynamic error message injection

1.2 Task Description

Refactored the previously developed User Directory application to utilize modern async/await syntax, replacing the older Promise chain approach. The primary objective was to master the implementation of try...catch blocks to manage asynchronous operations more cleanly. Additionally, I purposefully configured the application to request a broken API endpoint to test the robustness of the error-handling logic, ensuring that the application gracefully displays a user-friendly fallback UI instead of crashing when network or server-side errors occur.

1.3 Implementation Details

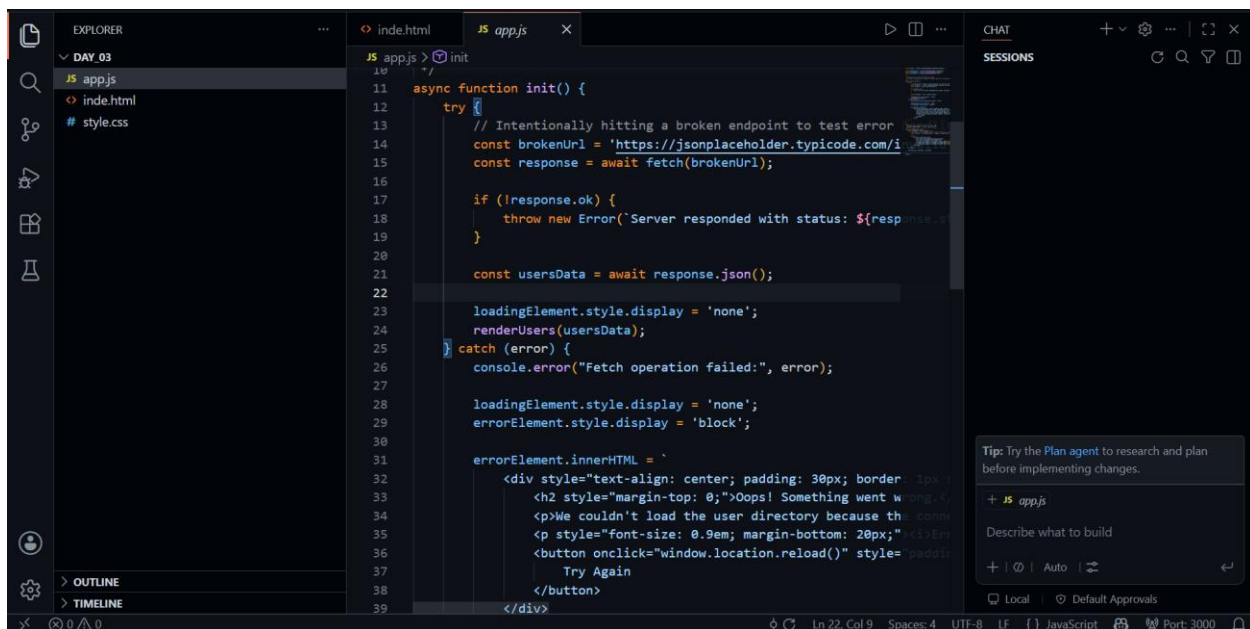
- Converted the primary init function to an async function, allowing for the use of await for cleaner, more readable asynchronous code execution.
- Wrapped the entire data-fetching logic inside a try block to isolate potential operations that might fail.
- Implemented a validation check on the response.ok property to explicitly catch and throw errors for non-200 HTTP status codes, such as 404 responses.

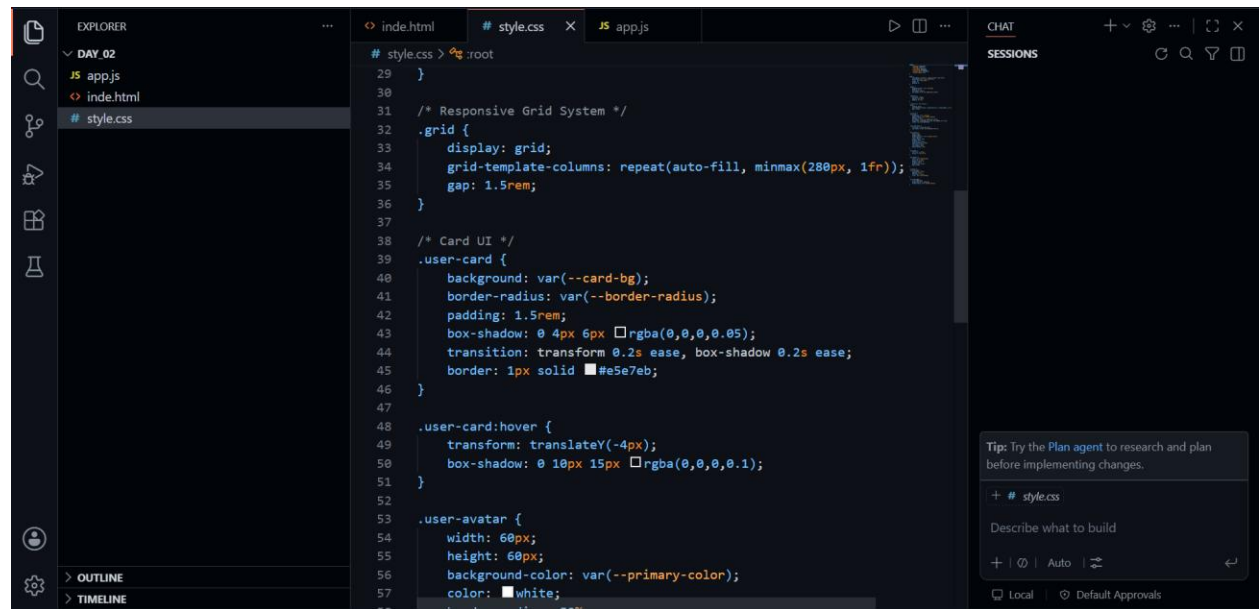
- Utilized a catch block to intercept both network failures and manually thrown HTTP errors, preventing script crashes.
- Designed a dynamic fallback UI that is injected into the DOM upon error detection, providing the user with specific information about why the data could not be loaded.
- Configured the application to target a non-existent API endpoint specifically to demonstrate and verify the functionality of the graceful error recovery system.
- Maintained the existing responsive CSS architecture to ensure that the error messages remain visually consistent with the rest of the application design.

1.4 Outcome

The Day 3 task was successfully completed, resulting in a cleaner and more maintainable codebase through the adoption of async/await. The project achieved its goal of professional-grade error handling by demonstrating that the application can identify and gracefully report failures when interacting with remote APIs. This implementation proves that the interface is resilient and capable of maintaining a positive user experience even when back-end services are unreachable or return errors.

Screenshots:





Screenshots:

